

NL2SQL Server — How It Was Built

Problem statement, technical approach, and the exact step-by-step process followed to take this from a blocked prototype to a simple, fully offline, working system.

Stack — Node.js · TypeScript · Express · PostgreSQL · Ollama (local model) **Date** — July 3, 2026

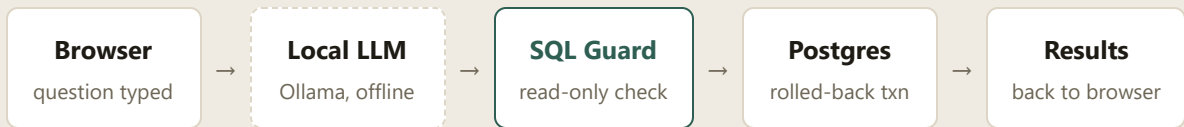
0	173	15 / 15	\$0
External API calls	Tables introspected	Tests passing	Spent to run it end-to-end

1. Problem statement

Answering a question about organizational data normally requires knowing SQL and the database's schema — a skill most people asking the question don't have.

This project builds a server that takes a plain-English question, automatically figures out the right SQL query grounded in the real database schema, safely executes it, and returns the results — without the user ever touching SQL, and without any part of the system calling out to an external server.

2. Technical approach



Every step runs on the local machine. The only network call the server ever makes is to localhost:11434 (the Ollama process) — nothing reaches the internet, and there is no API key anywhere in the codebase.

3. Step-by-step workflow followed

1 Verified the existing foundation

Confirmed schema introspection (173 tables read correctly) and direct database query execution both worked before changing anything.

2 Attempted a live end-to-end call

Request reached the Claude API correctly but returned `400 credit balance too low` — confirmed as a billing issue, not a code defect, since the request shape/auth were accepted.

3 Diagnosed a billing/product mismatch

Funds had gone to a Claude Pro chat subscription (claude.ai) — a different product from API credits (console.anthropic.com), which don't share billing. Decided to avoid further API spend entirely.

4 Installed a free local model runtime

Installed Ollama and pulled `qwen2.5-coder:7b` (~4.7GB) — an open-weight model that runs fully offline, no account or key required.

5 Wired the local model into `generateSql()`

Implemented `generateSql()` to send the schema and question to the local Ollama model over HTTP and parse its JSON response, sharing the same safety guard and database layer as before.

6 Found and root-caused a hallucination bug

First real run invented a nonexistent table. Cause: Ollama's default 2048-token context window silently truncated the 173-table schema before the model could see the right table.

7 Fixed it under hardware constraints

With only ~8GB RAM available, a larger context window wasn't practical. Instead, added a relevance filter that scores tables by keyword overlap with the question and sends only the ~20 most relevant.

8 Added a self-correction retry

If generated SQL still fails against the real database, the actual error is fed back to the model for one more attempt — applies to both LLM providers.

9 Verified end-to-end, twice

Two different real questions produced correct SQL and correct results against the live database, entirely offline, at no cost.

10 Simplified once the local model was proven reliable

Since only one database and one (offline) model are actually needed, removed the parts that existed only to support a cloud API and future multi-database "federation": the Claude integration, the provider switch, and the DataSource interface/router. Replaced them with one direct, plain Postgres module (src/db.ts) and one direct call to generateSql() — less code, same behavior, nothing left that could ever connect to an external server.

4. Outcome

QUESTION ASKED	SQL GENERATED	RESULT
List the 5 most recently added employees	SELECT name FROM pers.employee_master ORDER BY date_of_joining_org DESC LIMIT 5	5 correct employee names
How many employees are there in total?	SELECT COUNT(*) FROM pers.employee_master WHERE is_active = true	965 (correct)

The final codebase has no cloud AI dependency at all — @anthropic-ai/sdk was removed from the project entirely, and the only network call made at runtime is to the local Ollama process.

Full technical writeup: PROJECT.md · Setup instructions: README.md